

V

VÉRIFICATION DE MODÈLES

OU MODELCHECKING



INTRODUCTION

- Le model checking (Clarke et al., 1999) est une technique formelle qui permet de vérifier automatiquement le fonctionnement d'un système.
- Il a en entrée, d'une part, un modele qui décrit le comportement d'un système et, d'autre part, un formalisme qui spécifie les propriétés attendues du système.
- Il consiste à explorer l'espace des états du modele et à vérifier la conformité des évolutions des états vis-à-vis des propriétés attendues.



INTRODUCTION

- Il permet ainsi de détecter les erreurs de conception avant l'implémentation réelle du système.
- Ainsi, le modele checking permet de vérifier formellement si un modelé satisfait sa spécification et fournir, le cas échéant, un contre exemple.
- La technique du model checking a été largement appliquée dans plusieurs domaines.



INTRODUCTION

- Son intérêt a été particulièrement bien montré pour la vérification des protocoles de communication, les applications multimédia, et la sécurité informatique.
- Le modèle checking a pour avantage de fournir un contre exemple lorsqu'il détecte une violation de la propriété ce qui facilite la compréhension et la correction des erreurs.
- Grâce au model-checking, il a été prouvé l'exactitude d'une variété de protocoles de logiciels, notamment, des protocoles modernes d'e-commerce.



INTRODUCTION

- Cette technique a également permis de détecter des bugs dans les systèmes de réservation en ligne des compagnies aériennes.
- Elle a été utilisée également pour vérifier un module d'exécution du contrôleur de la sonde spatiale "Deep Space 1" en identifiant un défaut de conception majeur. Ce défaut n'a pas été détecté pendant la phase de test et a causé un blocage lors d'un vol expérimental de la sonde à 96 millions de kilomètres au-dessus de la Terre.



ÉTAPES DE VÉRIFICATION PAR MC

- La vérification de tout système en utilisant la technique de model checking passe par trois étapes :
 - Modélisation
 - Spécification
 - Vérification



MODÉLISATION

- Cette étape consiste à générer sous forme d'un modèle, une abstraction du système.
- Ce modèle doit être assez expressif pour pouvoir décrire sans ambiguïté les spécificités du système.
- Il doit également s'appuyer sur une sémantique rigoureuse permettant l'exécution et la vérification, à moindre coût, de propriétés à modéliser.
- Il s'agit de traduire le système sous forme d'une structure de données finie interprétable par un algorithme de model checking.
- Parmi ces structures de données : les automates, les réseaux de Petri, les diagrammes binaires de décision (BDD), etc.



SPÉCIFICATION

- Cette étape permet de traduire les requis (les propriétés attendues) d'un système dans un formalisme formel.
- On distingue différents types de requis : la sûreté, la vivacité, l'accessibilité, l'équité et l'absence de blocage, etc.
- Les logiques temporelles sont les formalismes les plus utilisés pour spécifier des propriétés.



VÉRIFICATION

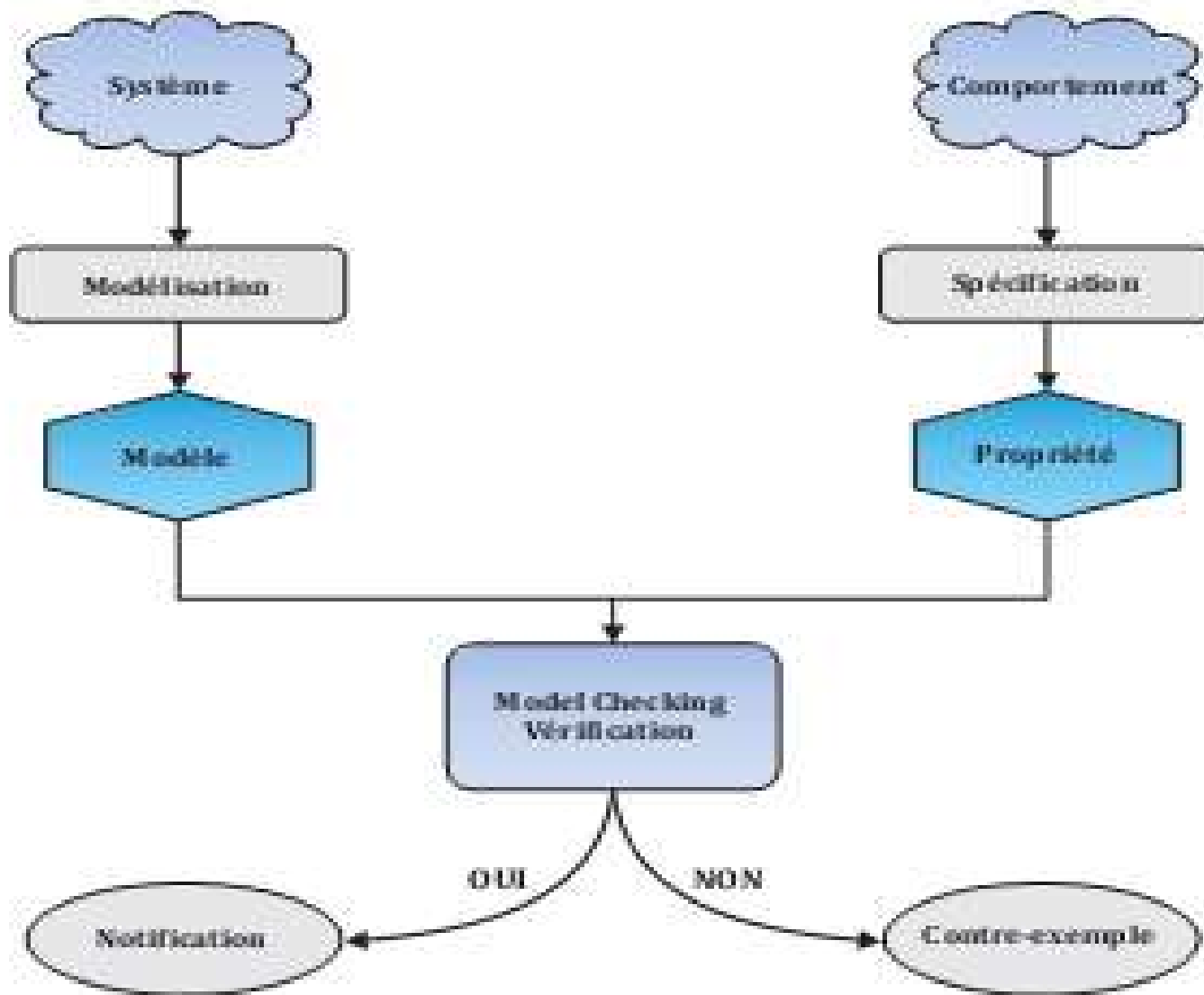
- Cette étape consiste à mettre en œuvre un algorithme qui prend en entrée un modèle (résultat de l'étape modélisation) et une spécification (résultat de l'étape spécification).
- Le modèle checker renvoie « vrai » si la spécification est vérifiée et fournit un contre exemple(trace d'erreur), le cas 'échéant.
- La vérification d'un modèle se base sur l'exploration de l'espace d'états et peut, donc, se heurter au problème d'**explosion combinatoire**.



VÉRIFICATION

- Dans ce cas, un retour aux étapes préliminaires (modélisation et spécification) est nécessaire pour modifier le modèle et la spécification afin de réduire le modèle et pallier ce problème majeur.
- Pour atténuer cette explosion combinatoire d'états, plusieurs techniques de vérification sont proposées : le model checking temporisé qui prend en considération l'aspect temporisé d'un système et le model checking statistique qui traite l'aspect probabiliste.





EXPLOSION COMBINATOIRE

- La principale limitation du model checking est le problème d'explosion combinatoire des états
- De nombreuses recherches ont été conduites pour trouver une solution à ce problème :
 - L'abstraction
 - Diagrammes de décision binaires (Binary Decision Diagram : BDD)
 - Model checking symbolique
 - Vérification à la volée
 - Modèle checking borné
 - Technique d'ordre partiel



L'ABSTRACTION

- Cette technique permet d'abstraire le modèle en ignorant quelques états du modèle, ce qui réduit le nombre de transitions et par conséquent la taille du modèle.
- Cette méthode d'abstraction est appelée "abstraction par restriction".
- Il y a aussi l'abstraction par "fusion d'états". Cette méthode consiste à regrouper un ensemble d'états ayant des caractéristiques communes en un seul état.



L'ABSTRACTION

- Il est également possible d'effectuer une suppression de certaines variables. Cette méthode est appelée "abstraction par variables".
- L'utilisation des automates observateurs constitue une autre méthode d'abstraction qui consiste à utiliser un automate observateur pour restreindre le comportement de l'automate sans toucher à sa structure.



DIAGRAMMES DE DÉCISION BINAIRES

- Les Diagrammes de décision binaires sont des structures de données qui représentent des fonctions booléennes sous forme d'un graphe dirigé acyclique.
- Ce graphe permet d'évaluer la valeur de vérité de la fonction booléenne en se basant sur les valeurs de vérité de ses variables.
- Les diagrammes de décision binaires sont très utiles dans la vérification formelle car ils permettent de représenter le comportement d'un système sous forme d'une formule ensembliste.



MODEL CHECKING SYMBOLIQUE

- Cette approche consiste à représenter symboliquement les états d'un modèle donné par des ensembles plutôt qu'individuellement.
- Chaque ensemble est représenté généralement par un **diagramme binaire de décision**.
- Le principe du model checking symbolique consiste à calculer des points fixes pour déterminer l'ensemble d'états accessibles ainsi que l'ensemble d'états satisfaisant une propriété donnée.



MODEL CHECKING SYMBOLIQUE

- Pour calculer les points fixes, le model checking symbolique détermine l'ensemble d'états prédécesseurs et l'ensemble d'états successeurs à un ensemble d'états donné.
- Il est plus adapté à la vérification des propriétés exprimées en logique temporelle CTL (Computational Tree Logic) puisqu'il calcule l'ensemble d'états d'une façon itérative.



VÉRIFICATION À LA VOLÉE

- Le principe de cette méthode consiste à calculer l'ensemble d'états accessibles du système et vérifier simultanément la spécification sur cet espace d'états.
- L'avantage de cette technique est qu'elle permet d'économiser la mémoire dédiée pour la vérification en stockant uniquement les traces parcourues plutôt que la totalité du graphe à explorer.



VÉRIFICATION À LA VOLÉE

- Cette méthode admet deux variantes de vérification :
 - i) **L'analyse en avant** qui consiste à calculer itérativement les successeurs des états initiaux et vérifier si l'état cible est calculé ou pas.
 - ii) **L'analyse en arrière** qui consiste à calculer les prédécesseurs des états cibles et vérifier si un état initial se trouve dans l'ensemble calculé.



MODÈLE CHECKING BORNÉ

- Modèle checking borné ou Bounded model checking : BMC
- Le modèle checking borné est une solution proposée pour atténuer le problème d'explosion combinatoire.
- Le principe de fonctionnement du BMC consiste à chercher, dans l'espace d'états du modèle, une exécution de longueur k violant la propriété à vérifier.



MODÈLE CHECKING BORNÉ

- Autrement dit, BMC consiste à chercher un contre-exemple de longueur k . Si la recherche aboutit alors la propriété est non satisfaite et le BMC génère le contre-exemple, sinon il faut augmenter la borne k jusqu'à trouver un contre-exemple sous la forme d'une exécution bornée de longueur k .
- Le model checking borné ne permet pas de valider une propriété donnée. Il permet plutôt de prouver qu'elle est non satisfaite en retournant un contre-exemple.



TECHNIQUE D'ORDRE PARTIEL

- Cette technique consiste à réduire la taille de l'espace d'états à explorer par un algorithme de vérification.
- Cette réduction est basée sur l'exploitation des différents ordonnancements des événements indépendants et concurrents éliminant ainsi des chemins d'exécutions redondants.
- Cette technique peut être considérée comme une abstraction par restriction.



FORMALISMES DE MODÉLISATION

- Plusieurs formalismes de modélisation du model checking sont proposés :
 - Les automates temporisés
 - Les automates temporisés probabilistes
 - Les automates temporisés de jeu.



AUTOMATES TEMPORISES

- Automates temporisés (Timed Automata) proposés par R. Alur et D. Dill en 1991.
- Il s'agit d'automates classiques munis de variables réelles positives ou nulles appelées horloges.
- Chaque automate possède un nombre fini d'états appelés locations et un ensemble fini de transitions étiquetées par des gardes ainsi que des actions de réinitialisation d'horloges.
- Chaque location est associée à une contrainte sur les horloges, appelée invariant qui permet de déterminer la durée de séjour dans un état avant de franchir la transition suivante.



AUTOMATES TEMPORISES

- Dans un état, une horloge évolue de manière continue et synchrone avec le temps.
- Sa valeur représente le temps écoulé depuis sa dernière mise à zéro.
- Un automate temporisé admet deux types de transitions :
 - i) Une transition d'action qui consiste à franchir une transition quand sa garde est vraie.
 - ii) Une transition de durée qui consiste à séjourner dans le même état avec l'évolution continue d'horloges en respectant l'invariant.

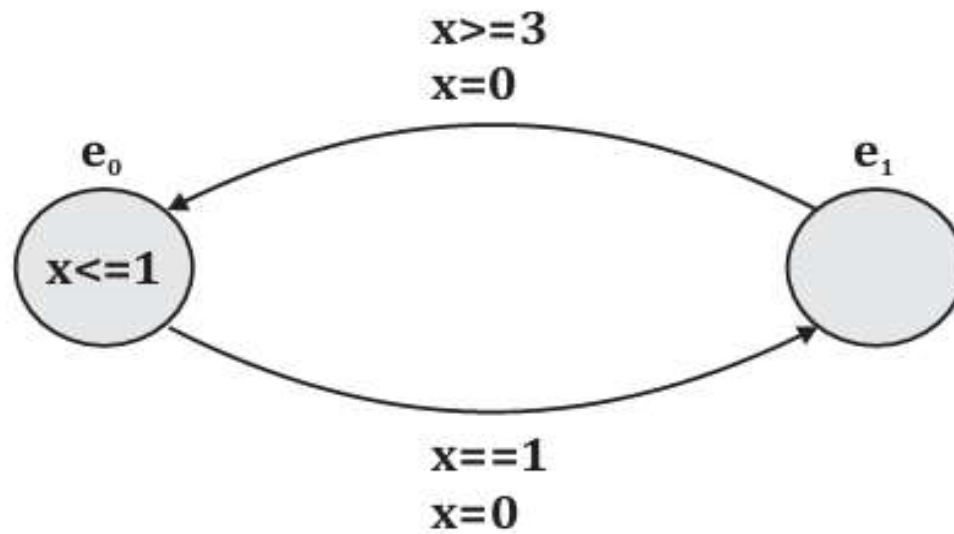


AUTOMATES TEMPORISES

- A titre illustratif, considérons l'automate temporisé simple, de la Figure suivante, composé d'une horloge x et deux états e_0 et e_1 .
- L'automate séjourne dans l'état e_0 jusqu'à une unité de temps. Quand l'horloge x est égale à 1, l'automate franchit instantanément la transition dont la garde est satisfaite ($x == 1$) et réinitialise l'horloge x .
- Puisque qu'il n'y ait pas d'invariant défini sur l'état e_1 , l'automate peut y séjourner au moins jusqu'à ce que la garde de sa prochaine destination soit satisfaite ($x \geq 3$). Autrement dit, l'automate doit séjourner au moins trois unités de temps dans l'état e_1 .



AUTOMATES TEMPORISÉS



AUTOMATES TEMPORISES

- Formellement, un automate temporisé A est défini par : $\mathbf{A} = (Q, q_0, X, \Sigma, I, T)$ où :
 - Q est un ensemble fini et non vide d'états,
 - $q_0 \in Q$ est l'état initial,
 - X est un ensemble fini d'horloges. Soit $C(X)$ l'ensemble de contraintes d'horloges appelées invariants, et 2^X l'ensemble de tous les sous-ensembles de X ,
 - Σ est un alphabet fini qui définit un ensemble d'actions,
 - $I : Q \rightarrow C(X)$ est une application qui associe à chaque état un ensemble de contraintes d'horloges de la forme $x \sim c$ où $\sim \in \{<, \leq, =, \geq, >\}$, c est une constante et $x \in X$.
 - $T \subseteq Q \times C(X) \times \Sigma \times 2^X \times Q$ est un ensemble fini de transitions. Chaque élément $t = \langle q, \varphi, a, r, q_0 \rangle \in T$ définit une transition de l'état q vers l'état q_0 en spécifiant sa garde φ , son action a , et le sous-ensemble d'horloges r à remettre à zéro suite à son franchissement.



AUTOMATES TEMPORISÉS À COÛT

- Les automates temporisés à coût sont une extension des automates temporisés classiques.
- Elles permettent une représentation plus efficace de l'aspect quantitatif d'un système temporisé.
- Les transitions et les états d'un automate temporisé à coût sont étiquetés par des informations additionnelles qui représentent, respectivement, le coût de franchissement d'une transition et le coût par unité de temps de séjour dans un état donné.



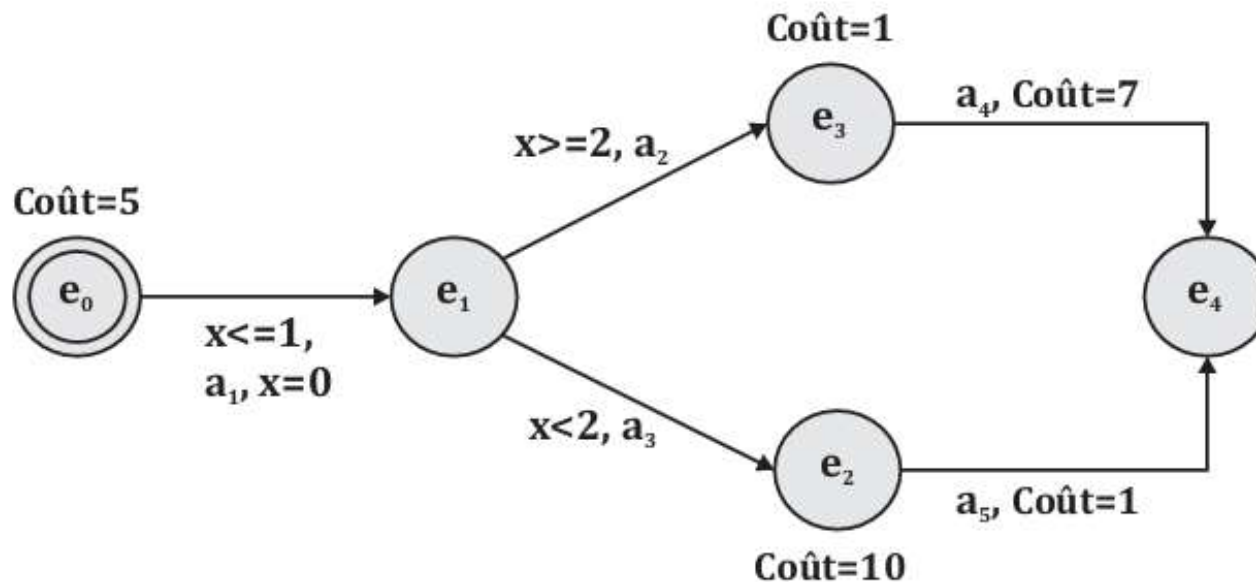
AUTOMATES TEMPORISÉS À COÛT

- Ainsi, chaque exécution de l'automate a un coût global, qui est le prix accumulé de tout franchissement d'une transition et tout délai dans les états le long du parcours.
- Ce modèle est particulièrement pertinent pour la modélisation de la consommation des ressources des systèmes "temps-réel". Ceci est fondamental dans le cadre des systèmes embarqués, qui sont confrontés à plusieurs contraintes de ressources (la bande passante, la mémoire, la consommation d'énergie, etc.).



AUTOMATES TEMPORISÉS À COÛT

- Considérons l'automate temporisé à coût A de la figure suivante.



AUTOMATES TEMPORISÉS À COÛT

- Les états e_0 , e_1 , e_2 et e_3 sont étiquetés par des coûts qui représentent le prix, par unité de temps, du séjour dans la location correspondante.
- Les transitions qui mènent vers l'état final e_4 sont étiquetées par des coûts qui correspondent respectivement aux prix de franchissement de ces deux transitions.
- Une exécution possible dans A est $E : (e_0, 1) \rightarrow (e_1, 1) \rightarrow (e_2, 3) \rightarrow (e_4)$.
 - Dans ce cas, l'automate séjourne dans e_0 une unité de temps. Le coût de séjour dans cette location est cinq par unité de temps.
 - L'automate passe, ensuite, à la location e_1 . Aucun coût n'est associé à cette location ou au franchissement de la transition $e_0 \rightarrow e_1$.



AUTOMATES TEMPORISÉS À COÛT

- Ensuite, la transition $e_1 \rightarrow e_2$ est franchie sans aucun coût additionnel.
- L'automate séjourne dans l'état e_2 trois unités de temps. Le coût de séjour dans cette location est dix par unité de temps, soit $(10 \times 3 = 30)$ unités de temps.
- Enfin, l'automate arrive à l'état final e_4 en franchissant la transition $e_2 \rightarrow e_4$ avec un coût d'une unité de temps.
- Par conséquent, le coût total associé à l'exécution E est $5 + 30 + 1 = 36$.



AUTOMATES TEMPORISÉS À COÛT

- Formellement, un automate temporisé à coût A est défini par : $A = (Q, q_0, X, \Sigma, I, T, P)$ où :
 - $(Q, q_0, X, \Sigma, I, T)$ est un automate temporisé,
 - Soit N l'ensemble des entiers positifs, $P : Q \cup T \rightarrow N$ est une application qui associe à chaque location un coût de séjour qui quantifie l'évolution d'horloge à cet état, et à chaque transition, un coût de franchissement.



AUTOMATES TEMPORISES DE JEU

- Automates temporisés de jeu (Timed Game Automata)
- Dans les automates temporisés, toutes les transitions franchissables peuvent se produire, sans aucun contrôle, ni restriction supplémentaire.
- Les automates temporisés de jeu permettent de distinguer deux classes de transitions : contrôlables et non contrôlables.
- Le franchissement d'une transition contrôlable peut être fixé, retardé ou encore bloqué, si besoin est. Par contre, aucun contrôle n'est possible sur les transitions incontrôlables.



AUTOMATES TEMPORISES DE JEU

- Les automates temporisés peuvent être considérés comme étant des automates temporisés de jeu où toutes les transitions sont incontrôlables.
- D'une façon générale, les automates temporisés de jeu permettent de formaliser des problèmes de contrôle des systèmes temporisés par un système de transitions dont l'évolution résulte de l'interaction de plusieurs agents (ou joueurs).



AUTOMATES TEMPORISES DE JEU

- Les transitions contrôlables sont associées à des conditions de déclenchement appelées aussi politiques ou stratégies.
- Le contrôleur recherché est ainsi l'ensemble de stratégies pour un agent (ou une coalition d'agents) permettant de résoudre le problème de contrôle et satisfaire l'objectif de contrôle correspondant quel que soit le comportement des autres agents.



AUTOMATES TEMPORISES DE JEU

- Le contrôleur peut être aussi défini comme un autre automate qui est synchronisé avec les transitions contrôlables de l'automate de jeu permettant d'y ajouter les conditions de déclenchement nécessaires afin de satisfaire l'objectif de contrôle.
- Généralement, pour un système temps réel, les transitions contrôlables sont activées par le contrôleur et les transitions incontrôlables, quant-à-elles, sont activées par l'environnement..

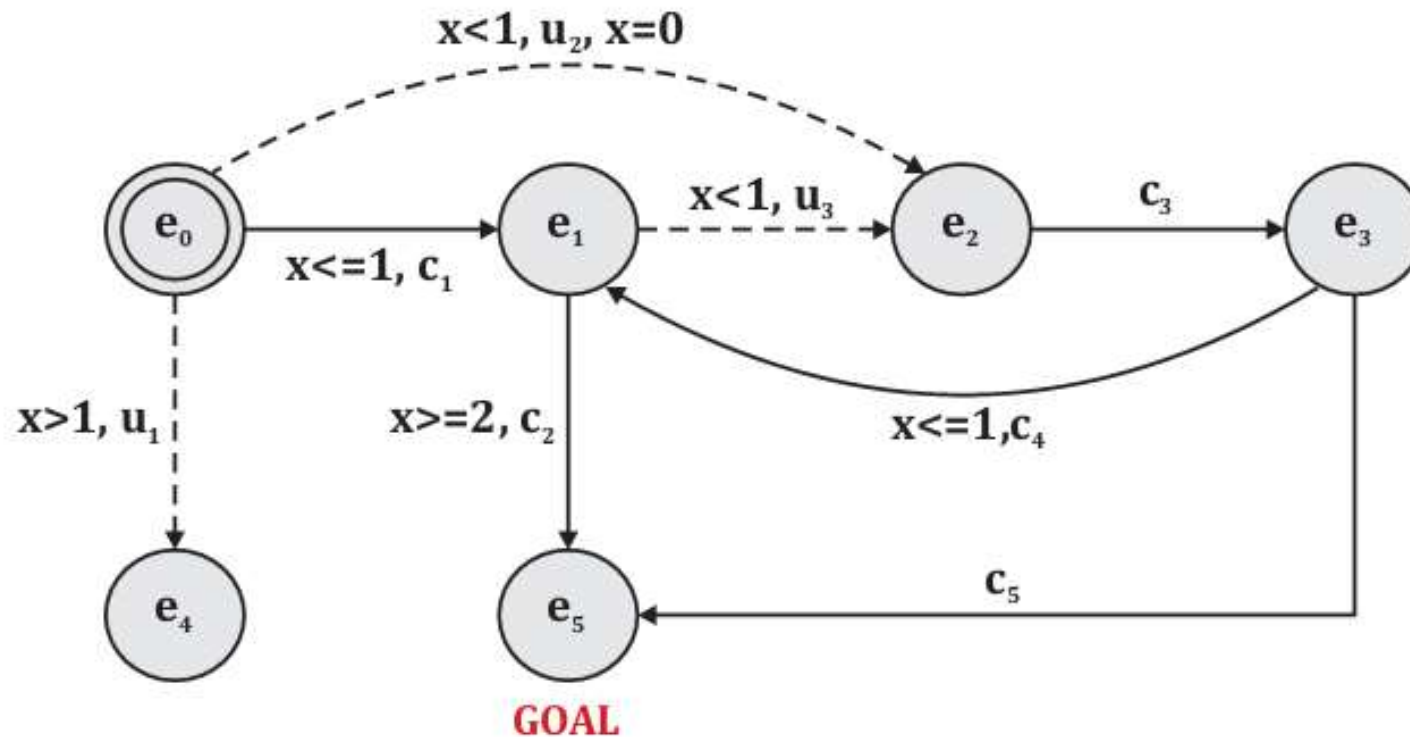


AUTOMATES TEMPORISES DE JEU

- Formellement un automate temporisé de jeu (jeu temporisé) est un 7-uplet $A = (Q, q_0, X, \Sigma_1, \Sigma_2, I, T)$, où :
 - $(Q, q_0, X, \Sigma_1 \cup \Sigma_2, I, T)$ est un automate temporisé,
 - Σ_1 et Σ_2 sont deux alphabets disjoints , Σ_1 représente l'ensemble des actions incontrôlables et Σ_2 représente l'ensemble des actions contrôlables.
- Considérons l'automate temporisé de jeux de la Figure suivante, où les actions incontrôlables (u_1, u_2, u_3) sont représentées par des traits pointillés et les actions contrôlables (c_1, c_2, c_3, c_4, c_5) sont représentées par des traits pleins.
- Dans cet exemple, il n'y a pas d'invariant sur l'état initial. L'automate peut alors décider de quitter l'état initial à tout moment.



AUTOMATES TEMPORISES DE JEU



AUTOMATES TEMPORISES DE JEU

- Si le contrôleur veut atteindre l'état "GOAL", il doit :
 - à partir de l'état e_0 , exécuter l'action c_1 avant que l'horloge x ne dépasse la valeur 1,
 - à partir de l'état e_1 , exécuter l'action c_2 si x atteint la valeur 2,
 - à partir de l'état e_2 , effectuer l'action c_3 , et à partir de l'état e_3 , choisir l'action c_5 , (il doit éviter l'action c_4),
- Nous venons de définir de façon informelle une stratégie de contrôle qui permet de forcer l'accessibilité de l'état cible "GOAL".



AUTOMATES TEMPORISES DE JEU

- Si le contrôleur souhaite jouer l'action c_1 lorsque l'horloge x est inférieure ou égale à 1, il peut franchir alors la transition correspondante uniquement si l'environnement n'a pas effectué auparavant l'action u_2 quand l'horloge x est strictement inférieure à 1.
- Si le contrôleur choisit de quitter l'état initial lorsque x vaut 1 et réussit à effectuer l'action c_1 , il arrive à l'état e_1 avec x égale à 1. Dans ce cas, il peut séjourner dans e_1 jusqu'à ce que l'horloge x soit supérieure ou égale à 2. Par la suite, il peut proposer l'action c_2 qui mène vers l'état cible "GOAL".



STRUCTURES DE KRIPKE

Une structure de Kripke d'un système de transition peut être vu comme un automate fini.

Une structure de Kripke est 5-uplet définie par :

$$K = (S, I, T, P, L)$$

S : ensemble d'états (*states*)

$I \subset S$: ensemble d'états initiaux

$T \subset S \times S$: relation de transition totale à gauche

$\forall s \in S. \exists s' \in E. sTs' \rightarrow$ chemins infinis pour T

$P = \{p, q, \dots\}$: ensemble de prédicats atomiques

$L : S \rightarrow 2^P$: fonction d'étiquetage

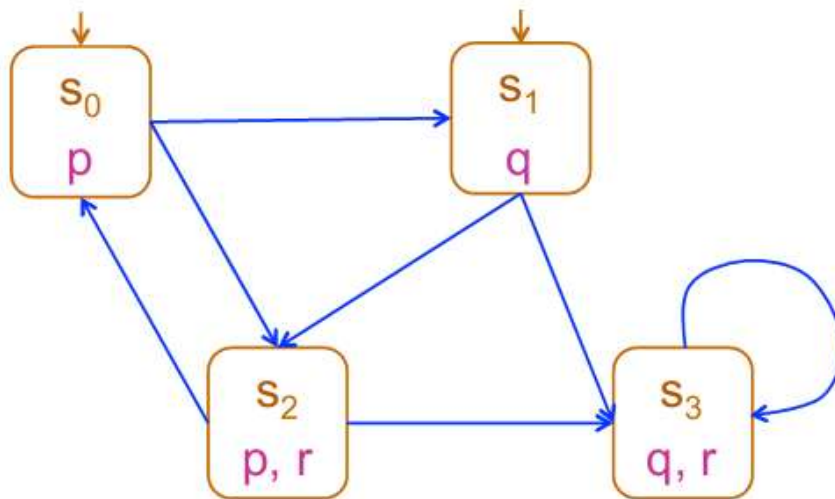
$L(s) = \{p, r, \dots\}$ ensembles des prédicats vrais en s

chemin : $\pi = s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots$

suffixe : $\pi[n] = s_n \rightarrow s_{n+1} \rightarrow s_{n+2} \rightarrow \dots$



STRUCTURES DE KRIPKE

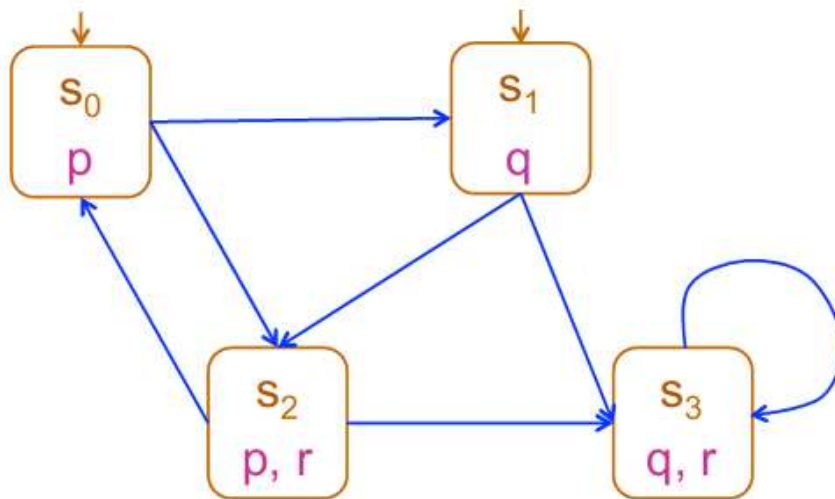


Propriétés satisfaites :

- P1 : p et q ne sont jamais vraies en même temps
- P2 : tout q est immédiatement suivi d'un r
- P3 : tout chemin infini depuis un état initial atteint r



STRUCTURES DE KRIPKE



Propriétés non satisfaites :

- P4 : tout p est immédiatement suivi d'un q
- P5 : tout chemin infini depuis un état initial atteint p



STRUCTURES DE KRIPKE

- Soit $M=(Q,T, A, q_0, AP, l)$ une structure de Kripke.
- Soit q un état. L'ensemble $\{q' \in Q \mid \text{il existe } a \in A, (q,a,q') \in T\}$ est l'ensemble des successeurs de q .
- On supposera que T est totale, i.e., chaque état a au moins un successeur.
- On peut compléter une structure de Kripke : on rajoute un état puits successeur des états dead-lock.



EXÉCUTIONS ET TRACES

- Soit $M = (Q, T, A, q_0, AP, l)$.
- Une exécution de M est une séquence infinie $r = q_0 a_0 q_1 a_1 q_2 a_2 \dots$ telle que $(q_i, a_i, q_{i+1}) \in T$, pour tout $i \geq 0$.
- On peut omettre l'étiquetage des transitions : $r = q_0 q_1 q_2 \dots$
- Une trace d'exécution de M est l'étiquetage d'une exécution : $l(r) = l(q_0)l(q_1)l(q_2) \dots$



LOGIQUE TEMPORELLE

- Généralement, le model checking utilise une logique temporelle afin de spécifier les propriétés décrivant les requis du système étudié.
- Une logique temporelle est un langage de spécification qui permet d'exprimer des propriétés faisant intervenir la notion d'ordonnancement dans le temps.
- Une logique temporelle permet d'exprimer des propriétés de type "après une instruction **i** un évènement **j** se produira" en utilisant des opérateurs temporels.
- On distingue principalement deux classes de logiques temporelles :
 - i) les logiques temporelles linéaires (Linear Temporal Logic : LTL),
 - ii) les logiques temporelles arborescentes (Computation Tree Logic : CTL)



LOGIQUE TEMPORELLE

- La logique temporelle linéaire permet de décrire le comportement des systèmes qui évoluent linéairement avec le temps.
- En effet, LTL permet d'exprimer des propriétés portant sur une séquence d'états donnée appelée exécution.
- Contrairement à LTL qui traite linéairement une seule trace d'exécution à la fois, CTL permet d'analyser plusieurs états futurs à partir d'un état donné du système en considérant plusieurs chemins d'exécution.



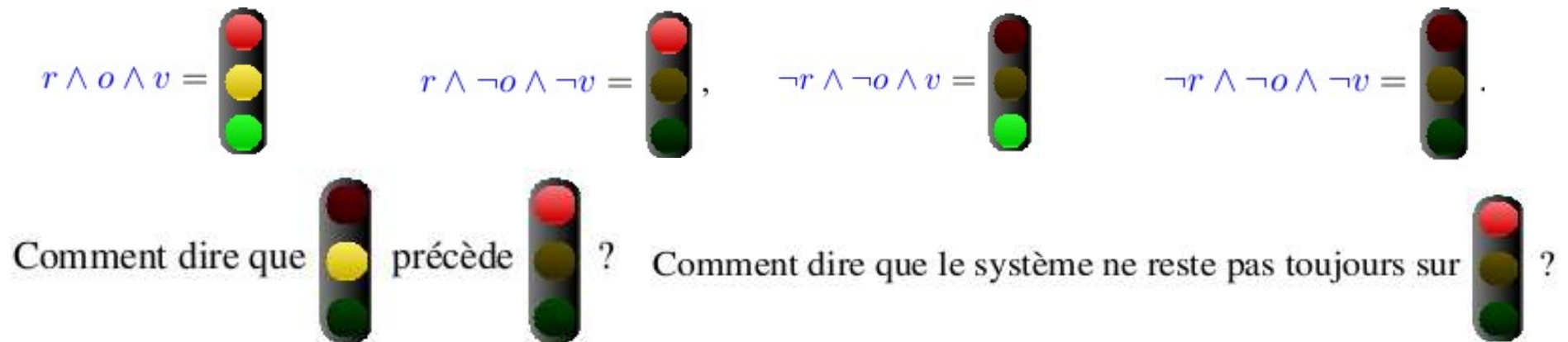
LOGIQUE TEMPORELLE LINÉAIRE (LTL)

- La logique temporelle linéaire ou LTL « Linear Temporal Logic (en anglais)» permet de représenter le comportement des systèmes réactifs au moyen des propriétés qui décrivent le système pour lesquels le temps se déroule linéairement
- En clair, on spécifie le comportement attendu d'un système, en spécifiant l'unique futur possible
- LTL est une extension de la logique propositionnelle qui permet de décrire l'évolution d'un système dans le temps.



LOGIQUE PROPOSITIONNELLE : L'INSTANT PRÉSENT

- La logique propositionnelle peut être utilisée pour caractériser un instant.
r : feu rouge allumé o : feu orange allumé v : feu vert allumé



- on a besoin de faire intervenir le temps



LOGIQUE MONADIQUE DU PREMIER ORDRE À UN SUCCESSEUR

- Les variables propositionnelles deviennent des prédicats unaires, paramétrés par le temps.
- $r(t)$, $o(t)$, $v(t)$: feux allumés à l'instant t
- $t + 1$: instant successeur immédiat
- $t \leq u$: ordre total sur les instants
- $\exists t, \forall t$: quantificateurs du premier ordre



LOGIQUE MONADIQUE DU PREMIER ORDRE À UN SUCCESSEUR

- $\neg \forall t. (r(t) \wedge \neg o(t) \wedge \neg v(t))$: le système ne reste pas tout le temps dans la configuration 
- $\forall t. ((\neg r(t) \wedge o(t) \wedge \neg v(t)) \Rightarrow (r(t + 1) \wedge \neg o(t + 1) \wedge \neg v(t + 1)))$: toute configuration  est immédiatement suivie de .
- $\forall t. \exists u. (t \leq u) \wedge (\neg r(u) \wedge \neg o(u) \wedge v(u))$: le système passe infiniment souvent par la configuration 



SYNTAXE DE LTL

Les formules de la logique temporelle linéaire LTL sont définies par la grammaire suivante :

- Les propositions atomiques : $\Phi, \Psi ::= p \mid q \mid \dots \mid \text{true} \mid \text{false}$ $\Phi, \Psi \in \text{LTL}$
- Les connecteurs booléens : Soit $\Phi, \Psi \in \text{LTL}$, toutes formules de la forme $\neg \Phi \mid \Phi \wedge \Psi \mid \Phi \vee \Psi \mid \Phi \Rightarrow \Psi \mid \Phi \Leftrightarrow \Psi$ appartiennent à LTL.
- Les connecteurs temporels : Soit $\Phi, \Psi \in \text{LTL}$, toutes formules de la forme $G \Phi \mid F \Phi \mid \Phi U \Psi \mid X \Phi$ appartiennent à LTL.

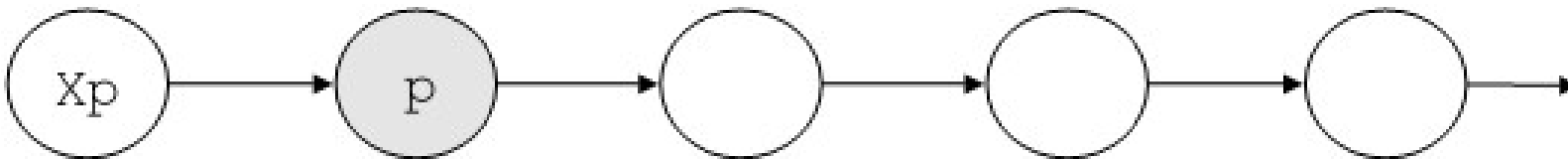


SÉMANTIQUE DE LTL

- Les connecteurs temporels permettent de parler du séquençement des états/événements observés le long d'une exécution.
- On introduit les connecteurs suivants : X (next), F (future), G (globally) et U (until).

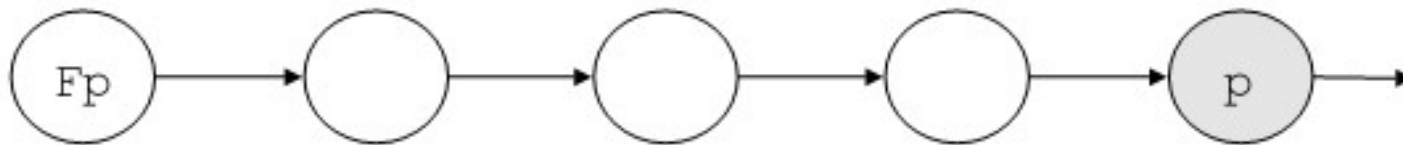
Soit p des propriétés atomiques des états observés.

- **X (next)** : Xp signifie que p est vrai dans l'état suivant le long de l'exécution

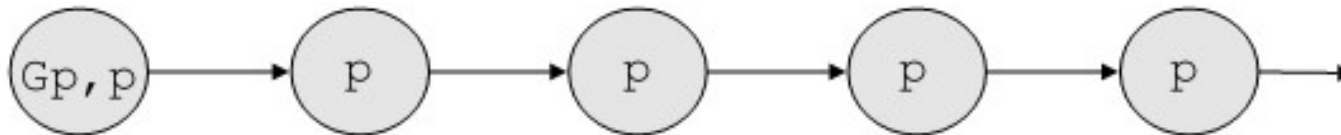


SÉMANTIQUE DE LTL

- **F (future)** : Fp signifie que p est vrai plus tard au moins dans un état de l'exécution; p sera vraie à un instant (présent ou futur)

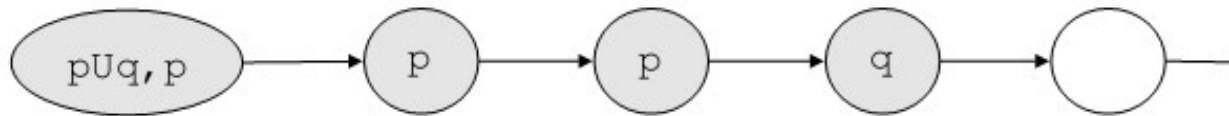


- **G (globally)** : Gp signifie que p est vrai dans toute l'exécution; p est vraie à tout instant



SÉMANTIQUE DE LTL

- **U (until)** : pUq signifie que p est toujours vrai jusqu'à un état où q est vrai



- On exprime des propriétés complexes le long d'un chemin en imbriquant les connecteurs. Voici quelques exemples de propriétés de chemin.
 - accessibilité (le long du chemin) : $F(x = 0)$
 - invariance (le long du chemin) : $G\neg(x = 0)$
 - vivacité (le long du chemin) : $G(p \rightarrow Fq)$
 - correction totale (le long du chemin) : $(\text{init} \wedge \text{precondition}) \rightarrow F(\text{end} \wedge \text{postcondition})$



LOGIQUE TEMPORELLE LINÉAIRE

- Équivalente à la logique monadique du 1^{er} ordre à un successeur
- Le système ne reste pas tout le temps dans la configuration

$$\neg \forall t. (r(t) \wedge \neg o(t) \wedge \neg v(t)) \iff \neg G(r \wedge \neg o \wedge \neg v)$$

- Toute configuration est immédiatement suivie de .

$$\forall t. ((\neg r(t) \wedge o(t) \wedge \neg v(t)) \Rightarrow (r(t+1) \wedge \neg o(t+1) \wedge \neg v(t+1))) \iff$$

$$G((\neg r \wedge o \wedge \neg v) \Rightarrow X(r \wedge \neg o \wedge \neg v))$$

- Le système passe infiniment souvent par la configuration

$$\forall t. \exists u. (t \leq u) \wedge (\neg r(u) \wedge \neg o(u) \wedge v(u)) \iff$$

$$GF(\neg r \wedge \neg o \wedge v)$$



SÉMANTIQUE DE LTL

- Une formule de LTL peut être satisfaite par une suite infinie d'évaluations de vérité des variables dans AP (ensemble fini de variables propositionnelles).
 - Soit $w = a_0, a_1, a_2, \dots$ tel un mot- ω . Soit $w(i) = a_i$. Soit $w_i = a_i, a_{i+1}, \dots$, qui est un suffixe de w . Formellement, la relation de satisfaction $\models$ entre un mot et une formule de LTL est définie comme suit :
 - $w \models p$ si $p \in w(0)$
 - $w \models \neg \psi$ si $w \not\models \psi$
 - $w \models \varphi \vee \psi$ si $w \models \varphi$ ou $w \models \psi$
 - $w \models X \psi$ si $w^1 \models \psi$ (ψ doit être vrai à l'étape suivante)
 - $w \models \varphi U \psi$ s'il existe $i \geq 0$ tel que $w^i \models \psi$ et pour tout $0 \leq k < i$, $w^k \models \varphi$ (φ doit rester vrai jusqu'à ce que ψ devienne vrai)
- On dit qu'un mot- ω w satisfait une formule LTL ψ quand $w \models \psi$.



SÉMANTIQUE DE LTL

- Les opérateurs logiques supplémentaires sont définis comme suit :
- $\varphi \wedge \psi \equiv \neg(\neg\varphi \vee \neg\psi)$
- $\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi$
- $\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$
- $\text{vrai} \equiv p \vee \neg p$, où $p \in AP$
- $\text{faux} \equiv \neg \text{vrai}$
- Les opérateurs temporels supplémentaires R, F et G sont définis comme suit :
- $\varphi R \psi \equiv \neg(\neg\varphi U \neg\psi)$
- $F \psi \equiv \text{vrai} U \psi$ (ψ devient éventuellement vrai)
- $G \psi \equiv \text{faux} R \psi \equiv \neg F \neg\psi$ (ψ reste toujours vrai)



Opérateur	Symbole alternatif ^{N 1}	Explication	Diagramme
Opérateurs unaires :			
$X \phi$	$\bigcirc \phi$	suivant : ϕ doit être satisfaite dans l'état suivant.	
$G \phi$	$\square \phi$	Globalement : ϕ doit être satisfaite dans l'intégralité des états futurs.	
$F \phi$	$\diamond \phi$	Finalement : ϕ doit être satisfaite dans un état futur.	
Opérateurs binaires:			
$\psi U \phi$	$\psi \mathcal{U} \phi$	Jusqu'à : ψ doit être satisfaite dans tous les états jusqu'à un état où ϕ est satisfaite, non inclus.	
$\psi R \phi$	$\psi \mathcal{R} \phi$	Réaliser : ϕ doit être satisfaite dans tous les états tant que ψ n'est pas satisfaite. Si ψ n'est jamais satisfaite, ϕ doit être vraie partout.	



ÉQUIVALENCES

- Soit Φ , ψ , et ρ des formules de LTL. Les tableaux suivants présentent des équivalences utiles.

Distributivité		
$X (\Phi \vee \psi) \equiv (X \Phi) \vee (X \psi)$	$X (\Phi \wedge \psi) \equiv (X \Phi) \wedge (X \psi)$	$X (\Phi U \psi) \equiv (X \Phi) U (X \psi)$
$F (\Phi \vee \psi) \equiv (F \Phi) \vee (F \psi)$	$G (\Phi \wedge \psi) \equiv (G \Phi) \wedge (G \psi)$	
$\rho U (\Phi \vee \psi) \equiv (\rho U \Phi) \vee (\rho U \psi)$	$(\Phi \wedge \psi) U \rho \equiv (\Phi U \rho) \wedge (\psi U \rho)$	

Négation		
$\neg X \Phi \equiv X \neg \Phi$	$\neg G \Phi \equiv F \neg \Phi$	$\neg F \Phi \equiv G \neg \Phi$
$\neg (\Phi U \psi) \equiv (\neg \Phi R \neg \psi)$	$\neg (\Phi R \psi) \equiv (\neg \Phi U \neg \psi)$	



ÉQUIVALENCES

Propriétés temporelles spéciales		
$F \Phi \equiv F F \Phi$	$G \Phi \equiv G G \Phi$	$\Phi U \psi \equiv \Phi U (\Phi U \psi)$
$\Phi U \psi \equiv \psi \vee (\Phi \wedge X(\Phi U \psi))$	$\Phi W \psi \equiv \psi \vee (\Phi \wedge X(\Phi W \psi))$	$\Phi R \psi \equiv \psi \wedge (\Phi \vee X(\Phi R \psi))$
$G \Phi \equiv \Phi \wedge X(G \Phi)$	$F \Phi \equiv \Phi \vee X(F \Phi)$	



EXERCICE

Exercices sur les connecteurs temporels :

1. – Fp est-il vrai si p vrai tout de suite dans l'état courant ?
2. – Gp est-il vrai si p faux dans l'état courant et vrai partout ailleurs ?
3. – pUq est-il vrai si p faux et q vrai dans l'état courant ?
4. – pUq est-il vrai si q est toujours faux, et p toujours vrai ?
5. – Faites un schéma d'une exécution où p est vrai infiniment souvent. Comparez avec vos camarades.
6. Qu'en déduisez-vous sur les langages naturels et graphiques ?



LOGIQUE TEMPORELLE ARBORESCENTE CTL

- Dans la LTL, on a considéré une seule exécution du système, et les connecteurs permettent de parler de propriétés le long de cette exécution. C'est une vision linéaire du temps : le futur est fixé.
- Cependant, on peut aussi voir le temps comme une structure branchante : à chaque état du système, plusieurs futurs sont possibles, selon l'action qui sera effectuée.
- On utilise des quantificateurs de chemins qui permettent de quantifier des propriétés sur les exécutions futures possibles à partir d'un état. On parle de propriétés d'états (d'un système). → logique temporelle arborescente



LOGIQUE TEMPORELLE ARBORESCENTE CTL

- La logique temporelle arborescente permet d'exprimer des propriétés portant sur les arbres d'exécution qui représentent l'évolution du système.
- Les formules CTL sont exprimées à l'aide du formalisme suivant :
 - Les propositions atomiques : p , q , . . . , $true$, $false$ qui s'interprètent directement sur un état.
 - Les connecteurs booléens : $\neg$ (négation), $\wedge$ (et logique), $\vee$ (ou logique), $\Leftrightarrow$ (équivalence), $\Rightarrow$ (implication).



LOGIQUE TEMPORELLE ARBORESCENTE CTL

- Les connecteurs temporels : $EF\ p \mid EG\ p \mid E\ p\ U\ q \mid EX\ p$
 $\mid AF\ p \mid AG\ p \mid A\ p\ U\ q \mid AX\ p$ avec E et A sont des quantificateurs de chemins.
- La syntaxe d'une formule CTL est alors la suivante : $\varphi, \psi ::=$
 $p \mid q \mid \neg\varphi \mid \wedge\psi \mid \varphi \vee \psi \mid \varphi \Leftrightarrow \psi \mid \varphi \Rightarrow \psi \mid EF\ \varphi \mid EG\ \varphi \mid E\ \varphi\ U\ \psi$
 $\mid EX\ \varphi \mid AF\ \varphi \mid AG\ \varphi \mid A\ \varphi\ U\ \psi \mid AX\ \varphi$ avec φ et ψ sont deux formules CTL.



LOGIQUE TEMPORELLE ARBORESCENTE CTL

Les formules CTL peuvent être interprétées comme suit :

- **EF φ** : Il existe une séquence d'états (opérateur E) qui mène à un état où la propriété φ est vérifiée (opérateur F).
- **AF φ** : Pour toute séquence d'états (opérateur A), il existe un état où la propriété φ est vérifiée (opérateur F).
- **EG φ** : Il existe une séquence d'états (opérateur E) où la propriété φ est toujours vérifiée (opérateur G).
- **AG φ** : Pour toute séquence d'états (opérateur A), la propriété φ est toujours vérifiée (opérateur G).
- **AX φ** : La propriété φ est vérifiée sur tous les états (opérateur A) immédiatement successeurs de l'état courant (opérateur X).



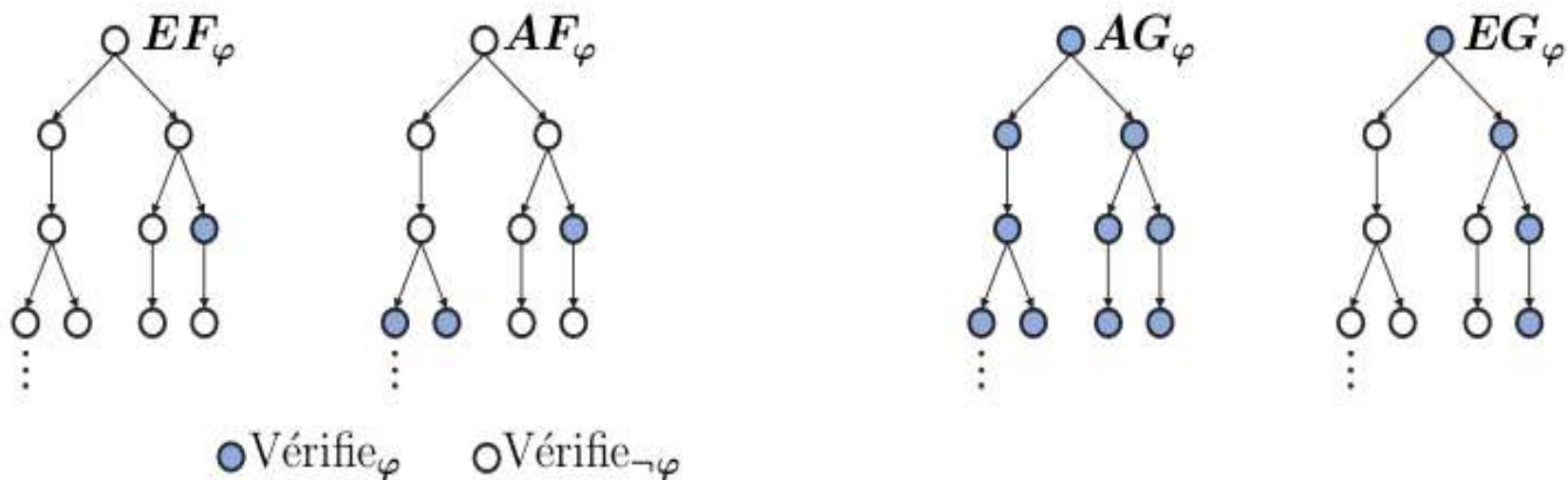
LOGIQUE TEMPORELLE ARBORESCENTE CTL

- **EX φ** : Il existe une séquence d'états (opérateur E) où il existe un état dont l'état successeur (opérateur X) satisfait la propriété φ .
- **A φ U ψ** : Pour toute séquence d'états (opérateur A) la propriété φ est vérifiée jusqu'à ce que la propriété ψ soit satisfaite.
- **E φ U ψ** : Il existe une séquence d'états (opérateur E) sur laquelle la propriété φ est vérifiée jusqu'à ce que la propriété ψ soit satisfaite.



LOGIQUE TEMPORELLE ARBORESCENTE CTL

Certaines formules CTL sont illustrées par la figure suivante.



EXERCICES

1. Dessinez des dépliages sur lesquels vous illustrerez les propriétés EX, AX, EU, AU.
2. Exprimer les propriétés suivantes :
 - a. Tous les états satisfont p .
 - b. On peut atteindre p par un chemin où q est toujours vrai.
 - c. Quelque soit l'état, on finit par revenir à l'état initial $init$.
 - d. Quelque soit l'état, on peut revenir à l'état initial $init$.
 - e. Absence de deadlock (partiel).



LOGIQUE TEMPORELLE ARBORESCENTE CTL

- Formellement, CTL est interprétée sur les états d'un système de transitions (qui caractérise les arbres issus de ces états).
- **Définition 1 (Système de transitions étiqueté S) :** Un système de transitions S est un tuple $(Q, q_0, A, \rightarrow, L)$ tel que :
 - Q est un ensemble (fini) d'états,
 - $q_0 \in Q$ est l'état initial,
 - A est un alphabet (fini) d'actions,
 - $\rightarrow \subseteq Q \times A \times Q$ est la relation de transition de S. On note $q \xrightarrow{a} q'$ si $(q, a, q') \in \rightarrow$,
 - $L : Q \rightarrow 2^{AP}$ où AP est un ensemble de propositions atomiques. $L(q)$ donne l'ensemble des propositions que satisfait l'état q .



LOGIQUE TEMPORELLE ARBORESCENTE CTL

- **Définition 2** (Chemin dans un système de transitions) : Un chemin dans un système de transitions est une séquence d'états $\sigma = q_0, q_1, q_2, \dots$ telle que $(q_i, q_{i+1}) \in \rightarrow$ pour tout $i \geq 0$.
- **Définition 3** (Sémantique de CTL) : Soit q un état de S . La sémantique de CTL est définie inductivement par :
 - $\phi \in AP, q \models \phi \iff p \in L(q),$
 - $q \models \phi \vee \phi' \iff q \models \phi \text{ ou } q \models \phi',$
 - $q \models \neg \phi \iff q \not\models \phi,$
 - $q \models AX\phi \iff \forall q' \text{ tel que } (q, q') \in \rightarrow, q' \models \phi,$
 - $q \models EX\phi \iff \exists q' \text{ tel que } (q, q') \in \rightarrow, q' \models \phi,$



LOGIQUE TEMPORELLE ARBORESCENTE CTL

- $q \models AX\phi \iff \forall \sigma \text{ un chemin tel que } \sigma(0) = q, \exists j \text{ tel que } \sigma(j) \models \phi,$
- $q \models EF\phi \iff \exists \text{ un chemin } \sigma \text{ tel que } \sigma(0) = q \text{ et } \exists j \text{ tel que } \sigma(j) \models \phi,$
- $q \models AG\phi \iff \forall \sigma \text{ un chemin tel que } \sigma(0) = q, \forall j, \sigma(j) \models \phi,$
- $q \models EG\phi \iff \exists \sigma \text{ tel que } \sigma(0) = q \text{ et } \forall j, \sigma(j) \models \phi,$
- $q \models A(\phi U \phi') \iff \forall \sigma \text{ tel que } \sigma(0) = q, \exists j \text{ tel que } \sigma(j) \models \phi' \text{ et } \forall k \text{ tel que } k < j, \sigma(k) \models \phi,$
- $q \models E(\phi U \phi') \iff \exists \text{ un chemin } \sigma \text{ tel que } \sigma(0) = q, \exists j \text{ tel que } \sigma(j) \models \phi' \text{ et } \forall k \text{ tel que } k < j, \sigma(k) \models \phi.$



EXTENSIONS DE LA LOGIQUE TEMPORELLE

CTL

- La logique temporelle s'est peu à peu étendue donnant naissance à d'autres logiques temporelles beaucoup plus expressives :
 - CTL*,
 - TCTL (Timed Computation Tree Logic)
 - PWCTL (Probabilistic Weighted Computation Tree Logic)
 - etc.



CLASSIFICATIONS DES PROPRIÉTÉS TEMPORELLES

- Il existe cinq grandes classes de propriétés pouvant être exprimées en logiques temporelles :
 1. Propriété de vivacité,
 2. Propriété de sûreté
 3. Propriété d'équité,
 4. Propriété d'atteignabilité (accessibilité) et
 5. Propriété d'absence de blocage.



PROPRIÉTÉ DE VIVACITÉ

- Une propriété de vivacité (liveness property) permet d'exprimer qu'un état ou un évènement va forcément se produire dans le futur.
- Exemple
 - – Quand une impression est lancée, elle finira par s'achever;
 - – Quand un message est envoyé, il finira par être reçu;
 - – Quand la pré-condition du programme est respectée, alors le programme termine
 - et la post-condition est respectée

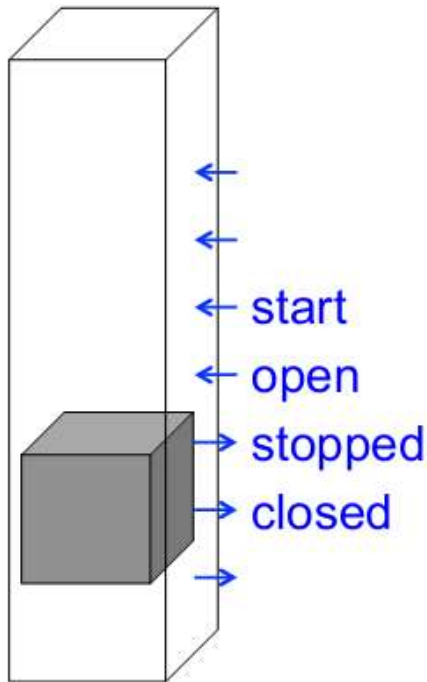


PROPRIÉTÉ DE VIVACITÉ

- Ce type de propriété peut être exprimé en logique LTL en utilisant l'opérateur F ('éventuellement') et l'opérateur U (jusqu'à).
- Cette propriété peut être également exprimée en logique CTL en utilisant l'opérateur AF.
- Exemple : $AF\ GOAL$, cette propriété permet de vérifier que le processus va atteindre fatalement son état cible GOAL.



PROPRIÉTÉ DE VIVACITÉ : EXEMPLE



- L'ascenseur ramasse les passagers qui l'appellent

A vérifier

- Après `call[i]`, l'ascenseur finit par s'arrêter à l'étage `i` et y ouvre sa porte avant de redémarrer

Condition d'environnement

- Quand il monte (rep. descend), l'ascenseur parcourt tous les étages dans l'ordre de la direction donnée, jusqu'à l'ordre d'arrêt (qui doit nécessairement arriver)

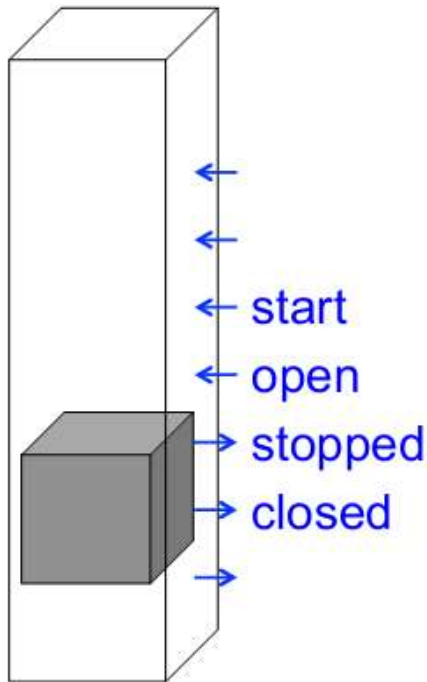


PROPRIÉTÉ DE SURETÉ,

- Propriété de sureté (safety property) : Une propriété de sureté permet d'exprimer que quelque chose de mauvais n'arrivera jamais.
- Exemple :
 - chaque fois que j'utilise un unlock, j'ai utilisé un lock avant;
 - Chaque fois que j'accède à mon compte, j'ai entré le bon code au préalable;
 - Quand la précondition du programme est respectée et que le programme termine
 - alors la postcondition est respectée.
- Cette propriété peut être exprimée en utilisant l'opérateur temporel G (toujours). Exemple : AG NOT FAIL
- Cette propriété permet de vérifier que le processus ne tombe jamais dans un état d'échec.



PROPRIÉTÉS DE SURETÉ (SAFETY)



- L'ascenseur ne peut pas voyager la porte ouverte.

A vérifier

- 1. Au départ, l'ascenseur ferme sa porte avant de démarrer
- 2. Après un start, open ne doit jamais arriver jusqu'à stopped
- 3. Après un open, start ne doit jamais arriver jusqu'à closed

Condition d'environnement

- l'ascenseur est initialement arrêté, porte ouverte



PROPRIÉTÉ D'ÉQUITÉ

- Propriété d'équité (Fairness) : Une propriété d'équité permet d'exprimer que quelque chose de bon se répète infiniment.
- Exemple :
 - si un processus demande son exécution, il finira par l'avoir.
- L'équité peut être exprimée en utilisant la logique temporelle LTL.
- Cependant, elle est non exprimable en utilisant la logique temporelle CTL puisque CTL ne supporte pas des opérateurs de type A ($FG\ p$). En effet, dans CTL, les opérateurs X, F, G et U doivent être précédés directement de A ou E.



PROPRIÉTÉ D'ATTEIGNABILITÉ

- Propriété d'atteignabilité (Reachability) : Une propriété d'atteignabilité permet d'exprimer qu'une certaine situation peut être atteinte.
- Exemples :
 - le compteur x peut prendre la valeur 0 (i.e. il existe un état atteignable où $x = 0$);
 - le point final du programme peut être atteint
- Cette propriété est moins forte que la vivacité qui exprime qu'une certaine situation doit forcément être atteinte.



PROPRIÉTÉ D'ATTEIGNABILITÉ

- La propriété d'atteignabilité peut être exprimée en utilisant l'opérateur temporel EF.
- Pour vérifier une propriété d'atteignabilité, il suffit de parcourir tout le graphe d'états et s'arrêter lorsqu'un tel état est atteint ou tous les états ont été explorés.

Exemple : EF PANNE, cette propriété exprime que le processus peut tomber en panne.



PROPRIÉTÉ D'ABSENCE DE BLOCAGE

- Absence de blocage (Deadlock freeness) : Une propriété d'absence de blocage permet d'exprimer qu'un système ne se bloque pas dans un état où il ne peut plus évoluer (aucune transition vers un autre état n'est possible).
- Exemple :
 - (non blocage total) Il existe au moins une exécution infinie du système;
 - (non blocage partiel) Il n'existe pas d'état bloquant.
- Une propriété d'absence de blocage peut être exprimée par la formule CTL : $AG (EX \text{ true})$.
- Une autre méthode pour conclure quant à l'absence de blocage est de vérifier que les états de blocage ne font pas partie de l'ensemble d'états accessibles.



MODEL CHECKERS

Les techniques de vérification par model checking sont classées en deux principales catégories :

- Les **modèles de vérification qualitatifs** : ce sont des outils de vérification de l'aspect qualitatif d'un système donné. Particulièrement, ils abstraient toute information quantitative sur les états du système tels que la durée de séjour dans un état donné et la durée qui sépare un état des autres. Il existe plusieurs modèles de vérification à aspect qualitatif tels que NuSM) , ASTRAL et SPIN.



MODEL CHECKERS

- Les **modèles de vérification quantitatifs** : Ce sont des outils de vérification permettant de quantifier l'aspect temps en utilisant des variables réelles appelées horloges.

Ils sont plus expressifs que les modèles de vérification qualitatifs puisqu'ils permettent de vérifier des propriétés temporisées.

Il existe plusieurs modèles de vérification à aspect quantitatif tels que UPPAAL,, KRONOS.



FIN
IT IS ENOUGH

